

FORGE — FDE Academy · Day 7 · 01-Jul-2026



Domain-Driven Design · Bounded Contexts · Ubiquitous Language

AI FDE - DDD - Introduction

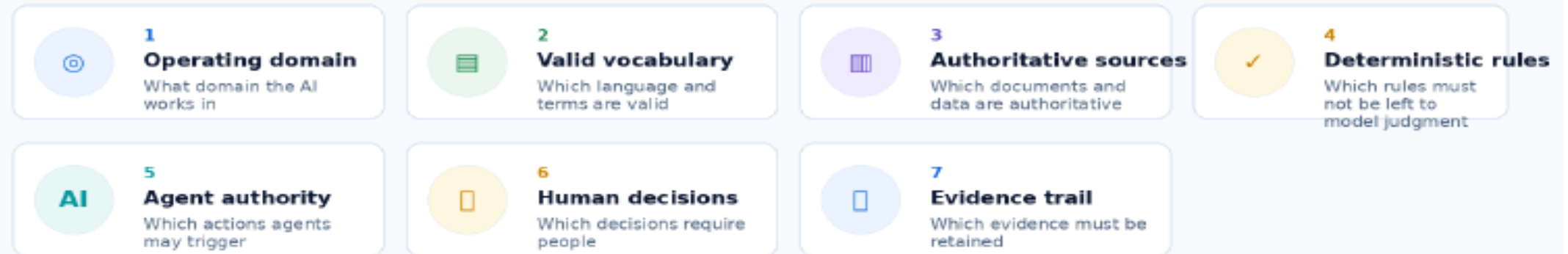
DDD for Gen AI FDEs

Why DDD is highly relevant for AI-forward deployed engineers

AI

Gen AI amplifies ambiguity. DDD gives the FDE a method to bound it.

What DDD defines for AI FDEs



From domain meaning to governed execution



Final Insight

DDD turns enterprise ambiguity into bounded, governed, explainable Gen AI workflows.

Metrics for DDD Success

How to know whether DDD implementation is actually working



Measure whether ambiguity, coupling, and change friction have been reduced.



Ambiguity ↓
Lower is better



Coupling ↓
Lower is better



Lead time ↓
Lower is better



Clarity ↑
Higher is better



Safety ↑
Higher is better

Metric area	Practical measure
 Language quality	Fewer disputed terms; glossary used in code, APIs, and stories
 Boundary quality	Fewer cross-context ownership conflicts
 Change velocity	Faster changes within one bounded context
 Defect reduction	Fewer defects from misunderstood rules
 Coupling	Fewer cross-context dependencies and data leaks
 Domain ownership	Clear owner for events, rules, APIs, and evidence
 AI readiness	RAG scope, agent authority, HITL, and evidence are bounded



Core Insight

DDD is successful when business change becomes easier and safer.

Derailment Factors to Avoid

Why DDD fails when patterns replace domain learning



DDD fails when teams copy patterns without doing real domain discovery.



Common derailments

- ✗ Treating bounded context as one microservice
- ✗ Building models without domain experts
- ✗ Starting with aggregates before subdomains
- ✗ Creating one enterprise-wide model
- ✗ Confusing CRUD tables with domain entities
- ✗ Applying CQRS or event sourcing everywhere
- ✗ Ignoring team ownership and operating model
- ✗ Measuring diagrams, not delivery outcomes



How to avoid them

- ✓ Start with domain conversations and context boundaries
- ✓ Use strategic DDD before tactical DDD
- ✓ Bring domain experts into discovery
- ✓ Validate boundaries through delivery and ownership
- ✓ Adopt advanced patterns only where complexity justifies them



Red Flags

Big-bang domain model

No domain expert involvement

Patterns applied by default

Architecture before learning

Low delivery impact



Core Insight

DDD is not diagramming. It is disciplined domain discovery.

When DDD Emerged

How Domain-Driven Design began and why it mattered



DDD was popularized by Eric Evans' 2003 book and created for complex business software.



01

2003 origin

Popularized through Eric Evans' book.



02

Purpose

Designed for complex domains with rich business rules and ambiguity.



03

Core idea

Model the domain around business meaning, not database structure.



04

Collaboration

Domain experts and developers build a shared language together.



05

Later influence

Became important for microservices, event-driven systems, and modernization.



Complexity



Shared language



Better models



Better systems



Core Insight

DDD was created to manage complexity caused by business ambiguity.

Why DDD Became Important

Why large systems outgrew one universal model



DDD became valuable when systems grew large, distributed, and semantically messy.



What changed

- ✓ Large systems had conflicting meanings
- ✓ Teams used the same terms differently
- ✓ Service boundaries became unclear
- ✓ Business rules leaked across systems



Critical correction

- ✓ Bounded Context is a model and language boundary
- ✓ It is not automatically one microservice
- ✓ It succeeds when meaning, ownership, and boundaries are explicit



From one tangled model...

One universal model tries to explain everything for everyone.

...to context-aligned models



Sales

Bounded Context



Billing

Bounded Context



Inventory

Bounded Context



Shipping

Bounded Context



Support

Bounded Context

Meaning

Ownership

Boundaries

Rules

Coupling



Core Insight

DDD became important when one universal model no longer matched business reality.

Public Success / Adoption Patterns

How DDD has been applied in public case reports



Use public cases carefully: most show experience patterns, not controlled ROI studies.

1

Fiverr Logo Maker



DDD used to manage complex product and design logic.

2

Xapo Bank



DDD ideas helped improve architecture and team alignment.

3

Kry / Primary Care



DDD used in healthcare modernization to move from spaghetti architecture toward cohesive domains.

Where DDD works best



High semantic complexity



Conflicting business meanings



Strong rules and workflows



Multiple teams and evolving boundaries

4

Wikimedia / Wikipedia



DDD explored for evolving a complex socio-technical system.

5

Azure guidance



DDD used for domain analysis and service boundary design.



Core Insight

DDD is strongest where complexity is semantic, not merely technical.

AI FDE x DDD - Anti-Patterns

Final Summary

Why non-DDD approaches are useful fragments but weak foundations



Gen AI FDEs should design from domain meaning first.

Comparison table

Non-DDD approach	Useful for	Weakness
• Technology-first	Fast prototype	Weak domain grounding
• Universal glossary	Shared vocabulary start	Hides context-specific meanings
• Data-first RAG	Source connectivity	Retrieves content without meaning
• Process-only	Workflow visibility	Misses state, rules, ownership
• Prompt-as-architecture	Interaction behavior	Cannot enforce governance

Design from domain meaning first

Meaning

Boundaries

Rules

Authority

Evidence



Final Insight

Tools, data, prompts, and process diagrams are not enough. Domain meaning must come first.

Prompt-as-Architecture Design

Why prompts alone are weak as a system design method



Prompting shapes behavior. DDD shapes the system that governs behavior.



Non-DDD pattern

- ✓ Put role, rules, policy, workflow, and controls into the prompt
- ✓ Overload the prompt
- ✓ Treat prompt text as architecture
- ✓ Let governance depend on model behavior



Why it is weak for Gen AI

- ✓ Business rules are not enforceable
- ✓ Context is not reliably bounded
- ✓ Tool access is not structurally governed
- ✓ Human review stays vague
- ✓ Evidence is not captured as a system object



Better stack



Core Insight

Prompts are the interaction layer. They are not the architecture layer.

Process-Only Design

Why workflow maps alone are weak for Gen AI



A workflow diagram shows sequence. A domain model explains meaning.



Non-DDD pattern

- ✓ Map the workflow and automate the steps
- ✓ Capture sequence but not meaning
- ✓ Miss entities, events, rules, exceptions, and ownership
- ✓ Give agents broad permissions



Why it is weak for Gen AI

- ✓ Exceptions are poorly modeled
- ✓ State changes are missed
- ✓ Rules mix with recommendations
- ✓ Ownership and approval points stay unclear
- ✓ Auditability stays weak



Process view → DDD-enhanced view

Process view	DDD-enhanced view
Step	Command
Status change	Domain event
Business object	Entity / aggregate
Rule	Policy / specification
Exception	Alternate domain path
Approval	Human decision point
Audit log	Evidence trail



Core Insight

Process maps show sequence. DDD shows meaning, state, rules, and responsibility.

Technology-First Design

Why starting with tools is weak for Gen AI system design



Technology-first design can produce fast prototypes, but weak production systems.



Non-DDD pattern

- ✓ Start with chatbot, RAG, copilot, or agent
- ✓ Lead with model, vector DB, prompt, or framework
- ✓ Discover domain workflow late
- ✓ Bury business rules inside prompts



Why it is weak for Gen AI

- ✓ Domain meaning stays unclear
- ✓ Rules, actions, and approvals are poorly defined
- ✓ Document validity and source authority are weak
- ✓ Architecture follows tools, not business complexity
- ✓ Demo success replaces operational success



AI FDE correction



Core Insight

A Gen AI system should not start with the model. It should start with the domain problem.

Universal Glossary Design

Why one enterprise-wide vocabulary is weak for Gen AI



One enterprise glossary hides context-specific meanings.



Non-DDD pattern

- ✓ Create one enterprise glossary
- ✓ Assume one word has one meaning everywhere
- ✓ Force false standardization across teams
- ✓ Ignore business context



Why it is weak for Gen AI

- ✓ Creates confusion across departments
- ✓ Makes RAG and agents overgeneralize
- ✓ Blurs context-specific rules and decisions
- ✓ Turns similar words into unsafe answers



Same term, different meanings

Term	Possible meanings
Customer	borrower, depositor, applicant, patient, member
Case	fraud case, AML case, clinical case, claims case
Risk	credit, fraud, clinical, privacy, operational
Approval	credit, KYC, discharge, prior authorization
Evidence	audit, transaction, clinical, legal



Core Insight

Gen AI needs bounded language, not generic vocabulary.

Data-First RAG Design

Why connecting everything to RAG is weak without domain boundaries



RAG without domain boundaries retrieves content, not necessarily meaning.



Non-DDD pattern

- ✓ Connect all documents, policies, databases, tickets, logs, and emails
- ✓ Treat data availability as domain understanding
- ✓ Retrieve by keyword similarity
- ✓ Mix old, draft, irrelevant, and wrong-context content



Why it is weak for Gen AI

- ✓ Source ownership becomes unclear
- ✓ Document authority and effective date are weak
- ✓ Jurisdiction and access rules get mixed
- ✓ Noise increases and answer quality drops
- ✓ Confident but unsafe synthesis appears



AI FDE correction

Design RAG around



Bounded context



Domain glossary



Source map



Policy hierarchy



Entity relationships



Retrieval intent



Access rules



Evidence requirements



Core Insight

RAG is not a document dump. It is controlled retrieval inside a domain boundary.

AI FDE - DDD - Outputs

End-to-End AI FDE Workflow Using DDD

From business problem to deployable Gen AI workflow



A deployable Gen AI workflow should follow a domain-first sequence.



Closing Insight DDD is the operating discipline for building domain-aware, governed Gen AI

DDD Outputs for AI FDE Delivery

The artefacts that make Gen AI deployable and governable



DDD produces concrete artefacts that make Gen AI deployable, governable, and testable.

DDD Artefact	AI FDE Use
 Domain Glossary	 prompt vocabulary, evaluator vocabulary, UI wording
 Bounded Context Map	 RAG boundary, agent scope, access control
 Context Map	 integration design and translation rules
 Event Flow	 workflow triggers and state changes
 Entity / Aggregate Model	 data model, memory model, tool contracts
 Rules Catalogue	 guardrails, rules engine, approval logic
 Agent Responsibility Map	 what the agent can read, draft, trigger, or escalate
 Human Review Map	 HITL checkpoints and approval gates
 Evidence Trail	 logs, citations, decisions, approvals, audit proof



Business domain



AI architecture



governed workflow



audit-ready evidence

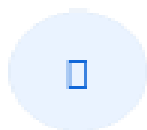


Core Insight

DDD is the bridge from business domain to governed AI delivery

Tactical DDD: Define What the AI Must Understand

Translate domain reality into precise design building blocks



Tactical DDD turns domain reality into precise design building blocks.



Entity

object with identity and lifecycle



Value Object

descriptive object without independent identity



Aggregate

consistency boundary around related entities and rules



Aggregate Root

main object controlling changes inside an aggregate



Domain Event

meaningful business event



Command

request to perform an action



Policy / Rule

business constraint or decision logic



Domain Service

domain logic not owned by one entity



Application Service

orchestrates workflow, tools, and integrations



Tells the AI system

- ✓ What exists
- ✓ What changed
- ✓ What is allowed
- ✓ What is prohibited
- ✓ What needs escalation
- ✓ What must be recorded



Core Insight

Tactical DDD gives AI structure, state, and decision discipline

What is Domain-Driven Design?

Designing systems around business meaning



DDD is the discipline of designing systems around business meaning, not technical assumptions.



Business reality



shared language



workflows



rules



decisions



systems



DDD helps teams model

- ✓ Business capabilities
- ✓ Domain vocabulary
- ✓ Workflows and events
- ✓ Rules and decisions
- ✓ Boundaries where meaning changes



For Gen AI FDEs

- ✓ Start with the domain model
- ✓ Clarify meaning before prompts
- ✓ Shape safe AI workflows
- ✓ Build for explainability and control



Core Insight

Meaning engineering before software engineering

Why DDD Matters for Gen AI FDEs

Domain clarity makes AI systems safer and more useful



Gen AI systems fail when domain meaning is vague.



Without DDD

- ✓ Ambiguous business terms
- ✓ RAG retrieves the wrong context
- ✓ Agents exceed their authority
- ✓ Rules mix with probabilistic reasoning
- ✓ Human review points stay unclear
- ✓ Audit evidence becomes weak

AI

With DDD

- ✓ Define the operating domain
- ✓ Standardize the working vocabulary
- ✓ Bound valid data and documents
- ✓ Separate rules from AI reasoning
- ✓ Set clear agent permissions
- ✓ Establish human approval points
- ✓ Retain traceable evidence



What the FDE really does



Clarifies meaning



Designs workflow boundaries



Governs AI behavior



Makes the system deployable

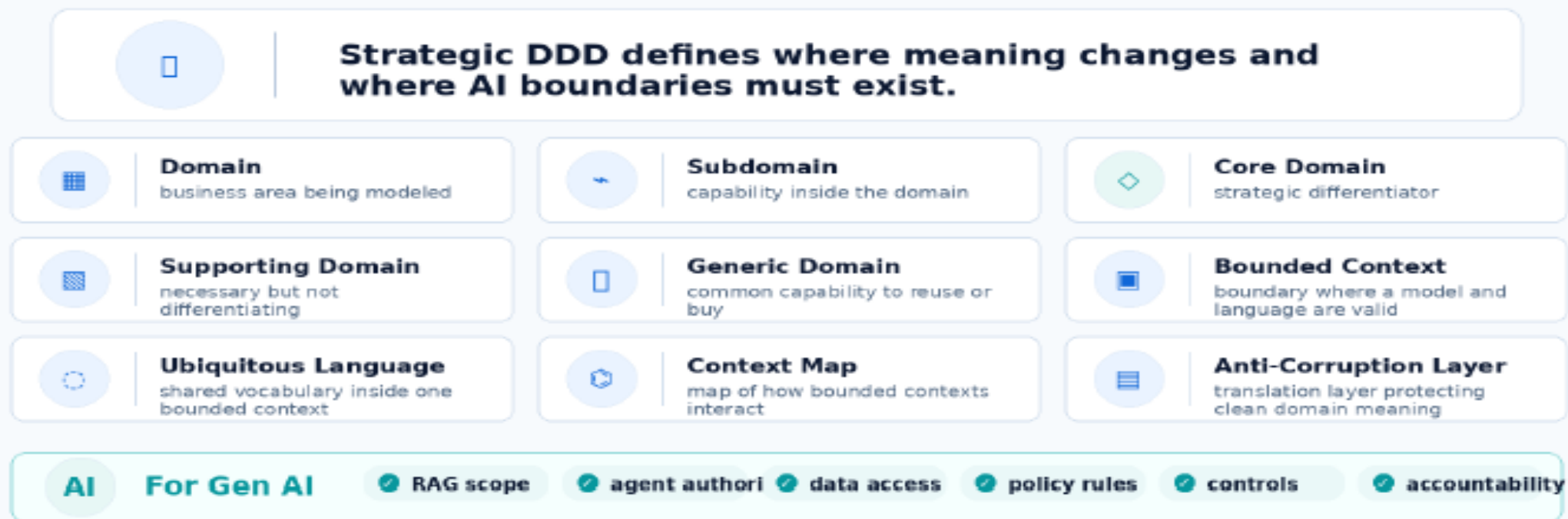


Core Insight

The AI FDE converts domain ambiguity into AI-ready operating logic

Strategic DDD: Define the Boundaries

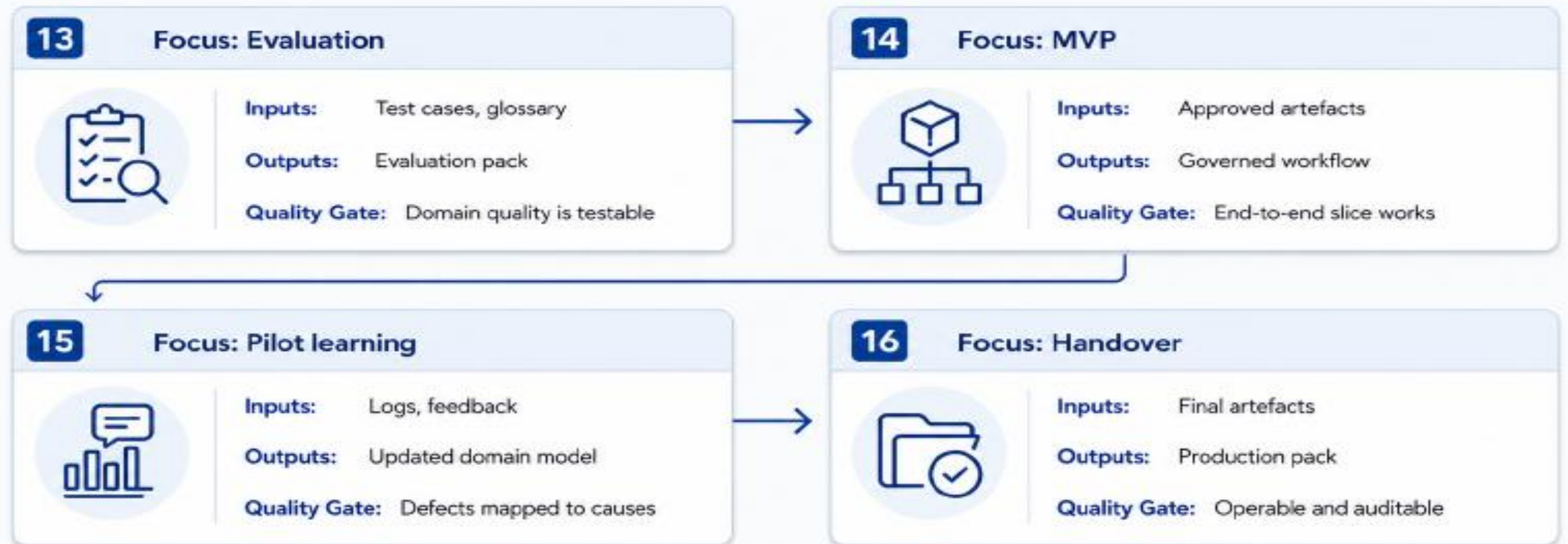
Where business meaning changes



Core Insight | **A bounded context becomes the safety boundary for AI**

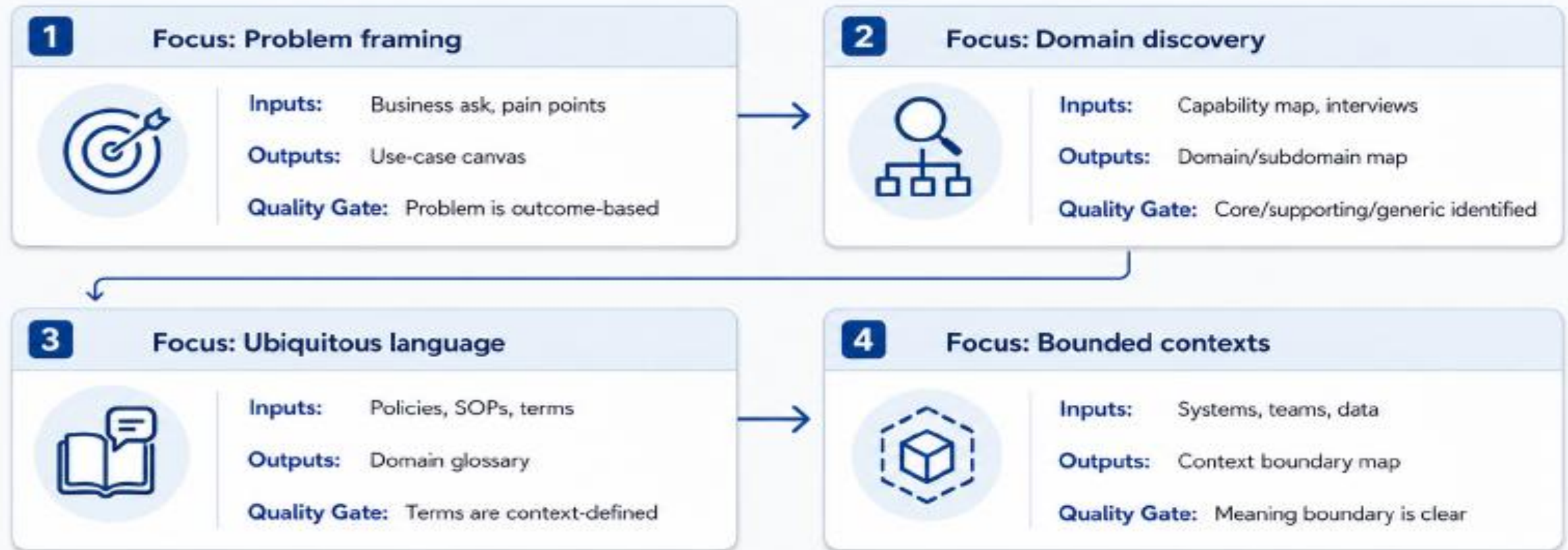
DDD Flow Stages 13–16

Testing, delivery, learning, and production readiness



DDD Flow Stages 1-4

Framing the problem and defining business meaning



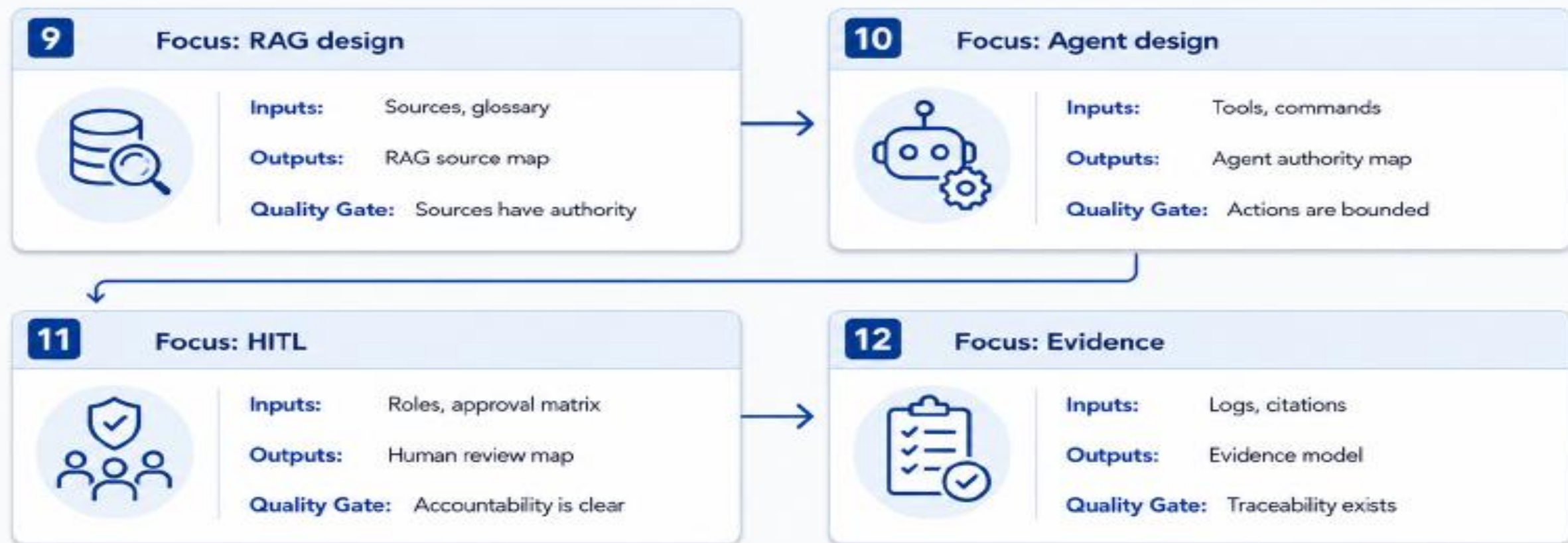
DDD Flow Stages 5–8

Linking contexts, events, and decision boundaries



DDD Flow Stages 9–12

Grounding retrieval, actions, oversight, and proof



AI FDE × DDD Implementation Workflow

Stage	DDD Focus	Inputs	Outputs	Quality Gate
1	Problem framing	Business ask, pain points	Use-case canvas	Problem is outcome-based
2	Domain discovery	Capability map, interviews	Domain/subdomain map	Core/supporting/generic identified
3	Ubiquitous language	Policies, SOPs, terms	Domain glossary	Terms are context-defined
4	Bounded contexts	Systems, teams, data	Context boundary map	Meaning boundary is clear
5	Context mapping	Integrations, handoffs	Context map	Translation rules exist
6	Event storming	Workflows, cases	Domain event map	Events are business facts
7	Tactical model	Events, rules, data	Entity/aggregate model	Consistency boundaries clear
8	Rules vs AI	Policies, regulations	Rules/AI matrix	Rules not hidden in prompts
9	RAG design	Sources, glossary	RAG source map	Sources have authority
10	Agent design	Tools, commands	Agent authority map	Actions are bounded
11	HITL	Roles, approval matrix	Human review map	Accountability is clear
12	Evidence	Logs, citations	Evidence model	Traceability exists
13	Evaluation	Test cases, glossary	Evaluation pack	Domain quality is testable
14	MVP	Approved artefacts	Governed workflow	End-to-end slice works
15	Pilot learning	Logs, feedback	Updated domain model	Defects mapped to causes
16	Handover	Final artefacts	Production pack	Operable and auditable

AI FDE's DDD Deliverable Stack

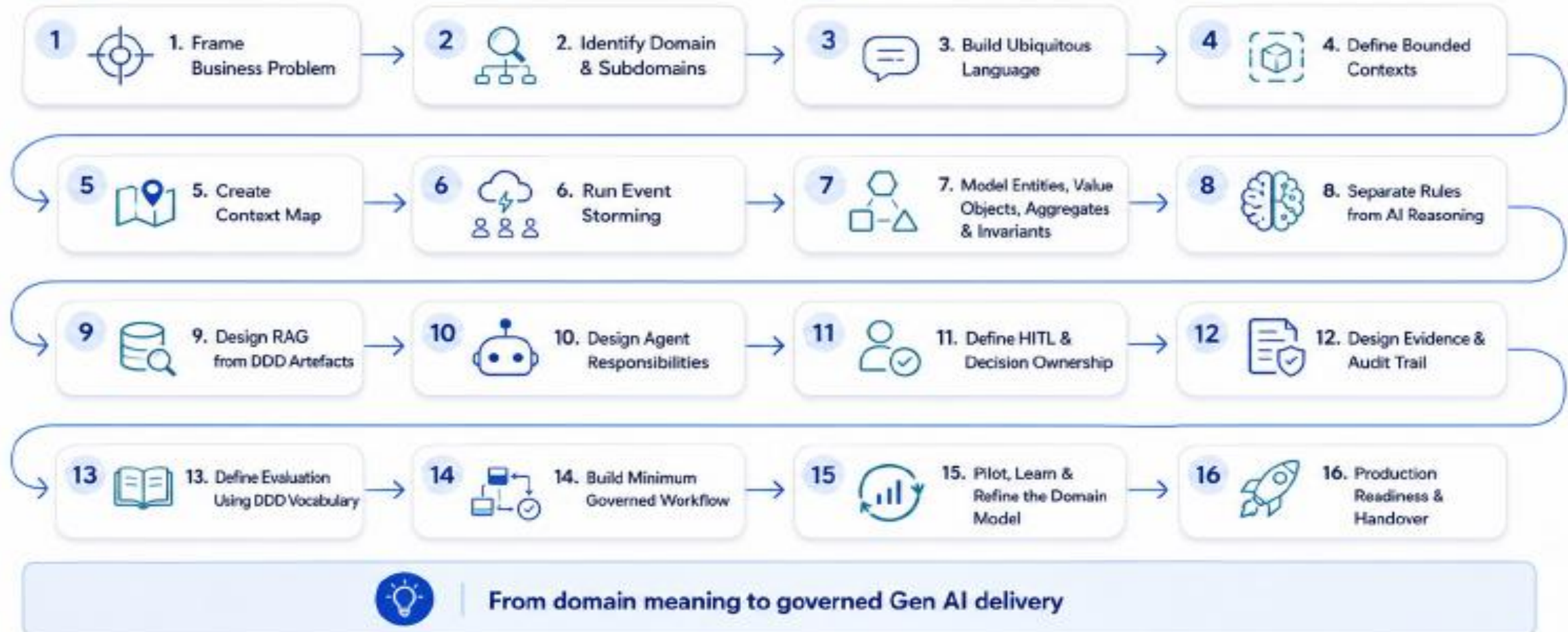
At minimum, produce these:

1. **AI Use-Case Canvas**
2. **Domain / Subdomain Map**
3. **Ubiquitous Language Glossary**
4. **Bounded Context Map**
5. **Context Map**
6. **Domain Event Map**
7. **Entity / Aggregate / State Model**
8. **Rules vs AI Reasoning Matrix**
9. **DDD-Grounded RAG Design Specification**
10. **Agent Responsibility and Authority Map**
11. **HITL and Decision Ownership Map**
12. **Evidence and Auditability Model**
13. **DDD-Based AI Evaluation Pack**
14. **Minimum Governed AI Workflow**
15. **DDD Feedback and Refinement Log**
16. **AI FDE DDD Production Handover Pack**

AI FDE x DDD - Stages

AI FDE × DDD Workflow Stages

Domain-Driven Design stages for Gen AI implementation



Meaning Flow for Enterprise AI



Why Gen AI Fails in Enterprises

